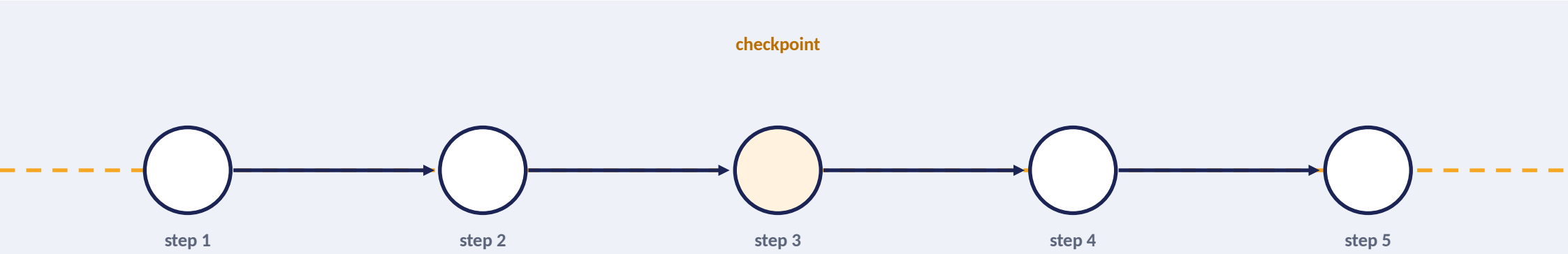
LANGGRAPH · CHAPTER 9 · FLASHCARDS

Checkpointing

Short-term memory that outlives a run

12 code cards on checkpoints, threads, resume and pauses



THE RULE

Checkpointer at compile, thread_id at invoke

Attach a saver once and name the conversation on every call; miss either and memory is gone

- Without a checkpointer every run restarts from your input: 1, 1, 1. With one and a fixed thread: 1, 2, 3.
- Seed once, then pass {}: count has no reducer, so {"count": 0} would overwrite the restored value.

PYTHON

```python
from langgraph.checkpoint.memory import MemorySaver

builder = StateGraph(State)                  # State: {"count": int}
builder.add_node("increment", increment)     # returns {"count": count + 1}
builder.add_edge(START, "increment")
builder.add_edge("increment", END)

graph = builder.compile(checkpointer=MemorySaver())       # 1. a place to save
config = {"configurable": {"thread_id": "counter-demo"}}  # 2. which save slot

for i in range(3):
    input_state = {"count": 0} if i == 0 else {}   # seed once, then restore
    result = graph.invoke(input_state, config)
    print(f"invoke {i + 1}: count = {result['count']}")
# invoke 1: count = 1
# invoke 2: count = 2
# invoke 3: count = 3

# builder.compile() with no checkpointer prints 1, 1, 1
```

Restore, merge, run

The checkpointer restores the thread and add_messages appends the new turn before the node runs

- The caller sends only the new message: the memory is the restored state, not the model.
- Resending the whole history by hand also works, but it buys memory only: no resume, no pauses.

PYTHON

```python
class State(TypedDict):
    messages: Annotated[list, add_messages]    # the reducer merges each turn

graph = builder.compile(checkpointer=MemorySaver())       # attach at compile
config = {"configurable": {"thread_id": "walid-session"}} # one conversation

hi = {"messages": [{"role": "user", "content": "Hi, my name is Walid"}]}
ask = {"messages": [{"role": "user", "content": "What is my name?"}]}

graph.invoke(hi, config)
result = graph.invoke(ask, config)
print("Bot:", result["messages"][-1].content)   # Bot: Your name is Walid!

# What the chat node received on the second invoke:
#   1. User:      Hi, my name is Walid        <- restored
#   2. Assistant: Hello, Walid!               <- restored
#   3. User:      What is my name?            <- merged by add_messages

saved = graph.get_state(config)    # peek at the latest checkpoint
print(saved.next)                  # () : the graph reached END
```

Your input merges through the reducers

On a resumed thread, the invoke input is merged into the restored state, field by field

- foo has no reducer, so every write replaces it; bar uses add, so it keeps growing across runs.
- Passing {"bar": []} does not reset bar: [] is added to the restored list, then the nodes append.

PYTHON

```python
from operator import add

class State(TypedDict):
    foo: str                              # no reducer: overwritten
    bar: Annotated[list[str], add]        # reducer: ["a"] + ["b"] = ["a", "b"]

def node_a(state: State) -> dict:
    return {"foo": "a", "bar": ["a"]}

def node_b(state: State) -> dict:
    return {"foo": "b", "bar": ["b"]}

# START -> node_a -> node_b -> END, compiled with InMemorySaver()
config = {"configurable": {"thread_id": "1"}}

graph.invoke({"foo": "", "bar": []}, config)
# {'foo': 'b', 'bar': ['a', 'b']}

graph.invoke({"foo": "", "bar": []}, config)   # same thread, same input
# {'foo': 'b', 'bar': ['a', 'b', 'a', 'b']}    <- bar doubled
```

One checkpoint per super-step

Each snapshot holds the values and the node to run next: next is the graph's position

- get_state(config) returns the latest snapshot; get_state_history(config) returns all, newest first.
- The input gets a checkpoint too: one invoke of this two-node graph saves four, two invokes save eight.

PYTHON

```python
history = list(graph.get_state_history(config))     # newest first
for step, snap in enumerate(reversed(history)):      # print oldest first
    print(f"checkpoint {step} (next={snap.next or 'done'}): {snap.values}")

# After invoke #1 of the foo/bar graph:
# checkpoint 0 (next=('__start__',)): {'bar': []}     <- the input
# checkpoint 1 (next=('node_a',)):    {'foo': '', 'bar': []}
# checkpoint 2 (next=('node_b',)):    {'foo': 'a', 'bar': ['a']}
# checkpoint 3 (next=done):           {'foo': 'b', 'bar': ['a', 'b']}

snap = graph.get_state(config)
snap.values           # what the graph knows now
snap.next             # which node(s) run next; () means finished
snap.config           # thread_id, checkpoint_ns, checkpoint_id
snap.metadata         # source ('input', 'loop', 'update') and step
snap.tasks            # work that is scheduled, interrupted or failed
snap.parent_config    # the checkpoint before this one
```

Replay or fork from a checkpoint

Hand an old snapshot's config back to invoke and the graph continues from that point

- A replay re-runs only the nodes after the checkpoint: double does not run again.
- A fork is update_state plus a replay; it adds checkpoints and never rewrites the old ones.

PYTHON

```python
# double (x2) -> add_ten (+10), compiled with MemorySaver()
config = {"configurable": {"thread_id": "time-travel-demo"}}
graph.invoke({"value": 5}, config)                    # {'value': 20}

history = list(graph.get_state_history(config))       # newest first
before_add_ten = next(s for s in history if s.next == ("add_ten",))
before_add_ten.values                                 # {'value': 10}

# Replay: continue from that checkpoint, unchanged
graph.invoke(None, before_add_ten.config)             # only add_ten runs: 20

# Fork: edit the saved value, then continue
forked_config = graph.update_state(before_add_ten.config, {"value": 100})
graph.invoke(None, forked_config)                     # {'value': 110}

# History was added to, not rewritten
graph.get_state(before_add_ten.config).values         # still {'value': 10}
len(list(graph.get_state_history(config)))            # 8
```

A thread is a conversation, not a person

Reuse a thread_id to continue a chat; change it to start one that knows nothing

- One checkpointer holds every thread, each sealed off; one user can own many threads.
- Facts that must follow a user into a new thread belong in the Store, keyed by user_id (Chapter 10).

PYTHON

```python
walid = {"configurable": {"thread_id": "walid/support-7"}}
sara  = {"configurable": {"thread_id": "sara/support-1"}}

graph.invoke({"messages": [("user", "My order is late")]}, walid)
graph.invoke({"messages": [("user", "What did I just report?")]}, walid)
# remembers: walid's thread restores the first message

graph.invoke({"messages": [("user", "What did Walid report?")]}, sara)
# isolated: sara's thread starts empty and knows nothing

# checkpointer + thread_id -> "where is this conversation?"
# Store + user_id          -> "what should future conversations know?"
```

Audit a loop with get_state_history

On a variable-length loop the history shows every pass, its score and where it went next

- Nine checkpoints for four nodes: the input and START get one, and every loop pass adds its own.
- In the repo, finalize sets approved=True even when the cap fires: check the score, not the flag.

PYTHON

```python
MAX_ITERATIONS = 3   # a hard stop so a stubborn low score can't loop forever

def route_after_analysis(state) -> Literal["revise", "finalize"]:
    if state["quality_score"] >= 8:            return "finalize"
    if state["iterations"] >= MAX_ITERATIONS:  return "finalize"
    return "revise"      # analyze bumps iterations, so the evidence is in state

builder.add_conditional_edges("analyze", route_after_analysis,
                              {"revise": "revise", "finalize": "finalize"})
builder.add_edge("revise", "analyze")          # loop back for re-analysis
graph = builder.compile(checkpointer=MemorySaver())

for snap in graph.get_state_history(config):   # newest first
    v = snap.values                            # the oldest snapshot is {}
    print(snap.next, v.get("processing_stage"), v.get("quality_score"))
# ()              finalized          9
# ('finalize',)   analysis_complete  9
# ('analyze',)    revision_complete  6
# ...             the score climbs 4 -> 6 -> 9; 9 checkpoints in all
```

Resume after a crash with invoke(None)

A crashed run keeps its finished steps; invoke(None, config) continues from next

- None means no new input: start at the saved next, not at START. step_one is not run again.
- A node can run twice if it dies before its checkpoint commits: guard side effects with idempotency keys.

PYTHON

```python
attempts_before_success = {"step_two": 1}   # fail once, then succeed

def step_two(state: State) -> dict:
    if attempts_before_success["step_two"] > 0:
        attempts_before_success["step_two"] -= 1
        raise RuntimeError("step_two failed (simulated)")
    return {"log": ["step_two"]}

# START -> step_one -> step_two -> step_three -> END; log uses add
graph = builder.compile(checkpointer=InMemorySaver())
config = {"configurable": {"thread_id": "resume-demo"}}

try:
    graph.invoke({"log": []}, config)
except RuntimeError as e:
    print(f"Graph crashed: {e}")

state = graph.get_state(config)
print(state.values["log"], state.next)   # ['step_one'] ('step_two',)

result = graph.invoke(None, config)      # <-- the resume idiom
print(result["log"])                     # ['step_one', 'step_two', 'step_three']
```

HUMAN-IN-THE-LOOP · PAUSE

Pause before a node with interrupt_before

interrupt_before runs up to a node, saves a checkpoint and hands control back to you

- While paused, get_state shows next = ('step_two',): the clearest sign the graph is waiting.
- get_state only lets you look: delete it and the resume is the same. invoke(None) picks the baton up.

PYTHON

```python
builder.add_edge(START, "step_one")
builder.add_edge("step_one", "step_two")
builder.add_edge("step_two", END)

graph = builder.compile(
    checkpointer=MemorySaver(),
    interrupt_before=["step_two"],        # planned pause, before this node
)
config = {"configurable": {"thread_id": "run-until-interrupt-demo"}}

result = graph.invoke({"log": []}, config)   # runs step_one, then STOPS
print(result)                                # {'log': ['step_one']}

snapshot = graph.get_state(config)           # inspect the frozen pause
print(snapshot.next)                         # ('step_two',)

final_state = graph.invoke(None, config)     # resume from the checkpoint
print(final_state)                           # {'log': ['step_one', 'step_two']}
```

HUMAN-IN-THE-LOOP · APPROVAL

An approval gate with update_state

Pause where the draft exists, collect the verdict outside the graph, write it in, resume

- The human lives between the two invokes: swap input() for a Slack button and the graph is untouched.
- update_state edits the paused checkpoint; next stays on review_decision, which reads the verdict.

PYTHON

```python
graph = builder.compile(
    checkpointer=MemorySaver(),
    interrupt_before=["review_decision"],   # after the draft, before the verdict
)

# 1. Run until the pause: create_draft runs, the graph freezes
result = graph.invoke(initial_input, config)

# 2. Outside the graph: plain Python shows the real draft, asks the human
decision = ask_for_review_decision(result["draft"])
# {"approved": False, "feedback": "Make it shorter."}

# 3. Write the human's decision into the saved checkpoint
graph.update_state(config, decision)

# 4. Resume: review_decision reads approved, the router picks the branch
final_state = graph.invoke(None, config)
# approved=True -> finalize       approved=False -> revise (uses feedback)
```

Same API on PostgreSQL

Swap MemorySaver for PostgresSaver and the thread outlives the process that wrote it

- Run setup() once to create the tables; then any process that knows the thread_id can continue it.
- MemorySaver: RAM, for tests. SqliteSaver: a local file. PostgresSaver: production, many workers.

PYTHON

```python
from langgraph.checkpoint.postgres import PostgresSaver

# Once, in 00_setup_tables.py: create the checkpoint tables
with PostgresSaver.from_conn_string(DB_URI) as checkpointer:
    checkpointer.setup()

# 01_save_name.py and 02_recall_name.py share this code; only message differs
THREAD_ID = "chat_session_walid"          # the only link between the two runs

with PostgresSaver.from_conn_string(DB_URI) as checkpointer:
    graph = build_graph(checkpointer)    # compile(checkpointer=checkpointer)
    config = {"configurable": {"thread_id": THREAD_ID}}
    result = graph.invoke({"messages": [HumanMessage(content=message)]},
                          config=config)

# Process A: "Hi! My name is Walid."   ... then the interpreter exits
# Process B: "What's my name?"         -> AI: Your name is Walid.
```

When the memory isn't there

A missing checkpointer fails quietly; real input on a crashed thread restarts the run

- No checkpointer: invoke runs and forgets; only get_state complains. Check the compile line first.
- After a crash, real input starts again from START and step_one runs twice. Resume with None.

PYTHON

```python
graph = builder.compile()                    # forgot the checkpointer
graph.invoke({"count": 0}, config)           # runs fine, remembers nothing
graph.get_state(config)
# ValueError: No checkpointer set

graph = builder.compile(checkpointer=MemorySaver())
graph.invoke({"count": 0})                   # forgot the thread_id: loud
# ValueError: Checkpointer requires one or more of the following
#   'configurable' keys: thread_id, checkpoint_ns, checkpoint_id

# step_two crashed on thread "resume-demo"; now pass real input:
graph.invoke({"log": []}, config)
# {'log': ['step_one', 'step_one', 'step_two', 'step_three']}
# Instead: graph.invoke(None, config) resumes at step_two, no repeat

# Checked on langgraph 1.2.6
```